Andromeda Spec 07: Integer Execution

Oct 10, 2026 · @Jerry

Part of the Andromeda spec set. Shared types and conventions are defined in file 00.

Modules: `alu`, `alu_bru` · **Milestone:** Int ALU/branch

1. What this unit does

Two integer execution ports. P0 (`alu`) is a single-cycle ALU. P1 (`alu_bru`) is the same ALU plus branch and jump resolution, and it also forwards CSR instructions to `csr_unit`. Both produce one result per cycle on their writeback bus.

2. How it works

2.1 ALU operations

ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR and AND, in register and immediate forms, plus LUI and AUIPC. By the time a uop reaches the ALU, `rr_stage` has already replaced rs2 with the immediate when `use_imm` is set, and supplied the PC for AUIPC. So the ALU only sees two 32-bit operands and an operation.

2.2 Branch resolution

Branches are *predicted* at fetch and *resolved* here. For each control-flow uop, `alu_bru` works out what really happens and compares it with what fetch assumed:

Instruction	Taken?	Target	Writes rd
BEQ, BNE, BLT, BGE, BLTU, BGEU	Compare rs1 with rs2	PC + imm	No
JAL	Always	PC + imm	PC + 4
JALR	Always	(rs1 + imm) with bit 0 cleared	PC + 4

The actual next PC (target if taken, PC + 4 if not) is compared with the predicted next PC (`pred_target` if `pred_taken`, otherwise PC + 4). If they differ, `br_mispredict` is set on the writeback bus. The ROB acts on it when the branch reaches the head: the branch commits, everything after it is flushed, and fetch restarts at the correct address.

Resolving here but recovering at commit means a mispredicted branch waits for every older instruction to finish before the core recovers. That's the simple v1 choice; early recovery with checkpoints is an open decision (file 00).

A taken target that isn't 4-byte aligned raises an instruction-address-misaligned exception (cause 0), reported through `wb_t` like any other exception.

2.3 Example

```
bne  x1, x2, loop     # predicted taken to "loop"
```

If x1 equals x2 at execute, the branch is actually not taken, so the actual next PC is PC + 4 while the predicted one was `loop`. `alu_bru` reports `br_taken = 0`, `br_target = PC + 4`, `br_mispredict = 1`.

When it commits, the ROB flushes and fetch restarts at PC + 4. The predictor is trained with the real outcome at the same time.

2.4 CSR forwarding

CSR instructions are handling Control Status Register (CSR), which affects the privilege levels, exception/interrupt handlers. CSR registers are not renamed and their writeback don't go through ROB, so it needs to be handled separately with all the csr related in and out ports.

3. Interface

3.1 alu (P0)

Signal	Dir	Width	Description
`flush`	in	1	`be_flush`
`in_valid` / `in_uop`	in	1 / uop_t	From rr_stage
`in_rs1` / `in_rs2`	in	32 / 32	Operands (rs2 already replaced by imm when use_imm)
`in_ready`	out	1	Always 1
`wb`	out	wb_t	Result next cycle
`wakeup`	out	wakeup_t	Destination tag, same cycle as wb in v1

3.2 alu_bru (P1)

Signal	Dir	Width	Description
`flush`	in	1	`be_flush`
`in_valid` / `in_uop` / `in_rs1` / `in_rs2`	in	1 / uop_t / 32 / 32	From rr_stage
`in_ready`	out	1	Low while csr_unit is busy
`csr_req_valid` / `csr_req_op` / `csr_req_addr` / `csr_req_wdata`	out	1 / 3 / 12 / 32	CSR access, only ever at ROB head
`csr_resp_valid` / `csr_resp_rdata` / `csr_resp_illegal`	in	1 / 32 / 1	Old CSR value, or illegal-access flag
`wb`	out	wb_t	Result and branch outcome (`br_valid`, `br_taken`, `br_target`, `br_mispredict`)
`wakeup`	out	wakeup_t	Destination tag

4. Implementation notes

- **One adder, many uses.** ADD, SUB, SLT and SLTU can share one 32-bit adder (subtract for compares). Branch compares need their own comparator so the branch target adder (PC + imm) can run in parallel.

- **Shifter.** A 32-bit barrel shifter is 5 levels of muxes. A common trick is to implement only a right shifter and reverse the bits for SLL.
- **Adder choice.** Let synthesis pick the adder structure first; Yosys and ABC do reasonably well. Only hand-build a carry-lookahead or parallel-prefix adder if the ALU is on the critical path after the first timing run.
- **Register the output.** The ALU is combinational between `rr_stage` and a writeback register. The writeback bus then drives the register file, the ROB, the bypass muxes and the wakeup comparators, which is heavy fanout; registering it keeps that off the ALU's path.
- **Share the code.** `alu_bru` should instantiate `alu` for its arithmetic, so both ports behave identically and are verified once.
- **Misprediction compare.** Compare full next-PC values, not just the taken bit, so a correct-direction but wrong-target JALR is caught too.

5. Verification checklist

- ☐ Every ALU operation against a software model with random operands, including shift amounts 0 and 31 and signed overflow
- ☐ Every branch type taken and not taken, predicted correctly and incorrectly
- ☐ JALR target with odd rs1 + imm clears bit 0
- ☐ Misaligned taken target raises cause 0
- ☐ Correct direction but wrong target (JALR) is flagged as mispredict
- ☐ CSR request/response, including illegal CSR, with `in_ready` low during the access
- ☐ Flush drops the uop in flight